part03_cli — 개발 환경과 adk CLI

`part03_cli` 폴더(= 에이전트들의 상위 폴더)에서 실행한다. 강의노트 **Part 3 · 03. 개발 환경과 CLI**(슬라이드 30–48)와 짝을 이룬다.

```
part03_cli/
├── my_agent/              # 📁 ADK 기본 에이전트 골격 (프롬프트 질의응답)
├── my_agent_mod/          # 📁 ADK 커스텀 에이전트 (날씨 조회 get_weather 도구 추가)
├── my-agents-cli/         # 📁 agents-cli 기본 엔터프라이즈 템플릿 (FastAPI, SSE, A2A, Terraform)
├── my-agents-cli-mod/     # 📁 agents-cli 커스텀 에이전트 (실시간 Open-Meteo 날씨 & 세계 시각)
├── run_direct_runner.py   # 🔧 직접 Runner 코드: 등록 → 실행 → 이벤트 소비 (슬라이드 23–25 · 39)
├── adk_internals.py       # 🟢/🔧 adk 가 대신 해 주는 5단계 재현 (슬라이드 31–38)
├── adk_cli_map.py         # 🟢 adk CLI 소스 구조 확인: 명령 트리 · AgentLoader · URI scheme · 경로 수 (슬라이드 32–36)
├── api_client.py          # 🔧 adk api_server 의 /run · /run_sse 호출 (슬라이드 43)
├── _path.py               # ⚙️ 상위 디렉터리 및 패키지 경로 탐색 헬퍼
├── README-KR.md           # 📄 전체 실습 한글 가이드 (본 파일)
└── README-EN.md           # 📄 전체 실습 영문 가이드
```

`.env` 는 실습 루트(`ADK_Labs/.env`) 하나만 두면 된다. adk가 에이전트 폴더에서 상위로 올라가며 찾는다.

📂 서브 폴더 개요 및 요약

서브 폴더	도구/프레임워크	핵심 내용	주요 실행 방법
`my_agent`	`adk` CLI	`adk create my_agent` 로 생성되는 가장 순수한 ADK 기본 에이전트. 도구 없이 Gemini 본연의 언어 모델 추론을 통해 질문에 답변.	`adk run my_agent` `adk web --port 8000`
`my_agent_mod`	`adk` CLI	기본 ADK 구조에 로컬 함수 `get_weather` 도구를 추가(Function Calling)한 도시 날씨 조회 에이전트.	`adk run my_agent_mod` `adk web --port 8000`
`my-agents-cli`	`google-agents-cli`	`agents-cli create` 로 생성된 프로덕션급 템플릿. FastAPI 기반 백엔드, SSE 스트리밍, A2A 프로토콜, Reasoning Engine 연동, 테스트(단위/통합/평가), Terraform 배포 코드 포함.	`cd my-agents-cli` `agents-cli playground` `agents-cli install`
`my-agents-cli-mod`	`google-agents-cli`	`my-agents-cli` 의 `app/agent.py` 를 고도화하여 실제 외부 Open-Meteo REST API(날씨/지오코딩) 및 IANA 표준 타임존 기반 실시간 세계 시각 조회를 지원하는 완성형 에이전트.	`cd my-agents-cli-mod` `agents-cli playground`

1) `my_agent/` 요약

- **구조**: `agent.py` , `__init__.py` , `.env` , `.gitignore` , `.adk/session.db`
- **역할**: 최소 단위의 ADK 기본 에이전트(`gemini-3.5-flash`). 별도 도구 없이 LLM의 기본 시스템 지침을 수행.
- **실행**: `part03_cli` 폴더에서 `adk run my_agent` 또는 `adk web --port 8000`

2) `my_agent_mod/` 요약

- **구조**: `agent.py` , `__init__.py` , `.env` , `.gitignore` , `.adk/session.db`
- **역할**: `get_weather(city: str)` 로컬 도구가 등록된 날씨 특화 에이전트(`weather_agent`).
- **실행**: `part03_cli` 폴더에서 `adk run my_agent_mod` 또는 `adk web --port 8000`

3) `my-agents-cli/` 요약

- **구조**: `app/` (FastAPI, agent, A2A, Reasoning Engine), `tests/` (unit, integration, eval), `deployment/terraform/`, `pyproject.toml`, `Dockerfile` 등
- **역할**: Mock 도구(`get_weather`, `get_current_time`)와 엔터프라이즈 기능(A2A 프로토콜, 평가 파이프라인, Terraform 인프라 배포)이 완비된 기본 템플릿.
- `uv run uvicorn app.fast_api_app:app --host 0.0.0.0 --port 8000`

4) `my-agents-cli-mod/` 요약

- **구조**: `my-agents-cli` 구조 + `app/agent.py` 실전 API 연동 + 웹 아키텍처 가이드 PPT
- **역할**: Open-Meteo API 실시간 날씨 및 전 세계 도시 시간 조회 기능 탑재, 오프라인 모드 (`AGENT_OFFLINE=1`) 폴백 제공.
- **실행**: `my-agents-cli-mod/` 내부에서 `agents-cli playground`

---

1. 같은 에이전트, 두 가지 실행 방식 (슬라이드 23–25 · 31–39)

ADK 응용 SW의 뼈대는 `Runner` 다. 같은 `my_agent/agent.py` 를 두 방식으로 실행해 비교한다.

방식	개발자가 쓰는 것	실습
직접 Runner 코드	root_agent + Runner · 서비스 · 입출력 코드	`run_direct_runner.py`
adk CLI	root_agent(agent.py)만	`adk run my_agent` · `adk_internals.py`

① 직접 Runner 코드: 등록 → 실행 → 이벤트 소비 (🔑)

```
uv run run_direct_runner.py "부산 날씨 알려줘"
```

```
① 등록   Runner(agent=weather_agent, session_service=InMemorySessionService)
② 실행   run_async(new_message='부산 날씨 알려줘')
   [Event 1] 모델 호출 → function_call get_weather({'city': '부산'})   (모델은 요청만)
   [Event 2] 도구 실행 → function_response get_weather: {'status': 'success', 'report': '흐림, 20°C'}
(Runner 가 실행)
③ 소비   최종 응답(Event 3): 부산의 현재 날씨는 흐리고 기온은 20°C입니다.
```

`run_async()` 한 번이 agent loop 한 바퀴다(슬라이드 25): 모델이 function_call을 "요청"하면 Runner가 도구를 실행하고 결과로 모델을 다시 부른다.

② adk 가 대신 해 주는 일 (🟢 키 없이 ①~④, 🔑 ⑤)

`adk run my_agent` 가 내부에서 하는 5단계를 ADK 자신의 코드(`AgentLoader` , `load_dotenv_for_agent` , `create_*_service_from_options`)로 그대로 재현한다.

```
uv run adk_internals.py                    # ①~④ (LLM 호출 없음)
uv run adk_internals.py --in_memory         # adk run --in_memory 와 같은 서비스 선택
uv run adk_internals.py --ask "서울 날씨?"   # ⑤ 까지: adk run 의 한 턴
```

```
① 찾기       AgentLoader → LlmAgent name='weather_agent' (from my_agent/agent.py)
② .env 로드   GOOGLE_API_KEY 설정됨=True · ADK_MODEL=gemini-3.5-flash-lite
③ 서비스 생성  session=PerAgentDatabaseSessionService → my_agent/.adk/session.db (SQLite, 첫 세션 때 생성)
              artifact=PerAgentFileArtifactService · memory=InMemoryMemoryService
④ Runner 생성  Runner(app='my_agent', root_agent='weather_agent')
⑤ 입구 제공   adk run 이면 여기서 input('[user]: ') 루프 → runner.run_async()
```

- **기본 세션 저장소는 InMemory가 아니다.** 플래그가 없으면 에이전트 폴더의 `.adk/session.db` (SQLite)에 저장된다. `--in_memory` 를 주거나 Cloud Run·Kubernetes에서 실행되면 InMemory로 바뀐다. 운영에서는 `--session_service_uri` 로 지정한다.
- `adk web` · `adk api_server` 도 같은 서비스·Runner를 만들고, 차이는 ⑤ 입구(HTTP)뿐이다(슬라이드 38).
- `google.adk.cli.utils` 는 ADK 내부 API라 버전마다 바뀔 수 있다(2.6.3 기준 학습용 재현).
- `adk run` · `adk_internals.py` 를 실행하면 `my_agent/.adk/` 폴더가 생긴다. 지워도 된다.

---

2. agents-cli (슬라이드 44–48, 상세는 Part 13 슬라이드 143–156)

agents-cli 소개 · adk와의 비교 · 명령 지도 · 프로젝트 구조 · 동작 구조 실습은 `part13_agents_cli/` 에 있다.

3. adk CLI 동작 구조 (슬라이드 32–36)

`google/adk/cli` 소스(google-adk 2.14.0 = GitHub main)를 슬라이드와 맞춰 직접 확인한다. 키 없이 실행된다.

```
python adk_cli_map.py           # commands · loader · services · routes
```

항목	슬라이드	확인할 것
`commands`	32	`adk` = `cli_tools_click.main` Click 그룹, 명령마다 구현 함수
`loader`	34	`AgentLoader.load_agent('my_agent')` → `LlmAgent`
`services`	34	`memory · gs · file`
`routes`	35–36	api_server 경로 22개 · adk web 59개(`/dev/*` 35개)

4. adk 주요 명령 (슬라이드 40–43)

명령	하는 일
`adk create my_agent2`	agent.py · __init__.py · .env 골격 생성
`adk run my_agent`	터미널 REPL (예: `서울 날씨 알려줘`)
`adk run --save_session --session_id s1 my_agent`	종료 시 `my_agent/s1.session.json` 저장 → eval 데이터로 재활용
`adk run --resume my_agent/s1.session.json my_agent`	저장한 대화를 이어서 진행
`adk run --replay queries.json my_agent`	`{"state": {...}, "queries": [...]}` 입력 파일을 사람 입력 없이 실행 (세션 파일이 아니다)
`adk web --port 8000`	개발 UI: http://localhost:8000 (Events · Trace · State · Graph 탭)
`adk api_server --port 8000 --session_service_uri="sqlite:///sessions.db"`	헤드리스 REST/SSE 서버

플래그는 버전마다 다를 수 있으니 `adk run --help` 로 확인한다.

---

5. api_server 호출

```
adk api_server --port 8000
```

다른 터미널에서:

```
uv run api_client.py
```

curl로 직접 호출할 때:

```
# 세션 생성
curl -X POST localhost:8000/apps/my_agent/users/u1/sessions/s1

# 에이전트 질의 실행
curl -X POST localhost:8000/run -H "Content-Type: application/json" -d "
{\"app_name\":\"my_agent\",\"user_id\":\"u1\",\"session_id\":\"s1\",\"new_message\":
{\"role\":\"user\",\"parts\":[{\"text\":\"서울 날씨?\"}]}}"
```